

Internet Programming

CSE343

Lab 6

Backend Basics, REST, Async/Await

Agenda

- The backend
- REST
- Let's design our own API
- How to integrate frontend with backend
- Async/Await

The backend

- The backend is the code that is concerned with the logic of the application
- As opposed to frontend which is concerned with displaying the application
- So far, we have been learning frontend
- Time to learn some backend!

The backend

So, what does the backend do?

Take Amazon as an example:

- when you first open the page, the backend sends you the products catalog to be displayed
- When you order an item, it starts processing your order: updating the stock, creating a delivery shipment, ... etc
- It is also responsible for processing the payment

So basically, the backend handles the logic of your application

The backend

- For frontend, we used html, css, and js which are the native languages that run in the browser.
- We also used a library like React to make our lives easier
- For backend, you can use any programming language.

The backend

- Here are some examples for some languages with some of their backend framework:
 - Python: FastAPI, Flask, Django
 - Java: Spring Boot
 - Ruby: Ruby on Rails
 - Go: Gin
 - Javascript: Express, NestJS
 - There are many many more of these ...
 - Essentially, any language had some dudes that said: "I wanna do backend in this language" and built a framework for it
- We will study some of these later

The backend

So, how do we approach such an important part of the webapp?

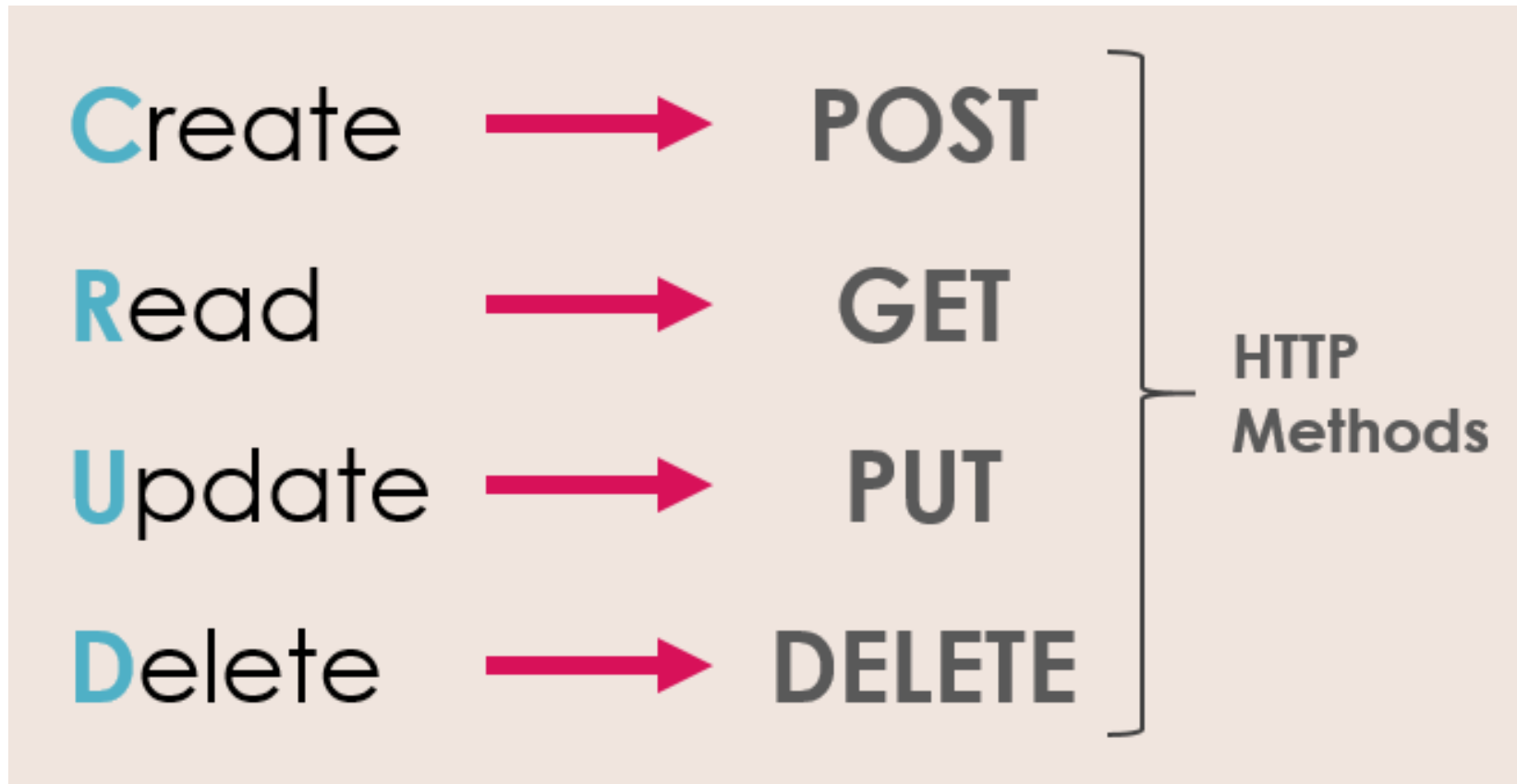
- We have our logic running in any general purpose programming language
- We expose the functionalities through an API
- There are a couple of API architectures out there
- We will focus on one of them since it's the most widely used
- REST!

REST

- For many applications, the role of the backend simply boils down to CRUD operations:
 - **C**reate: Inserting data to some database like making an order on amazon (where an order is just a simple entry in the orders table)
 - **R**ead: Reading the data from the database like getting the previously made orders
 - **U**ppdate: Updating the data like changing the address for an order
 - **D**eleete: Deleting the data like cancelling an order (where the cancellation is simply deleting the order row from the database)

REST

Rest uses HTTP methods to achieve these operations:



REST

Now we know the methods, what's next?

- We specify the name of the method, then the URL of the data we want to apply the method on.
- For example:
 - GET /api/orders -> returns all the orders we made
 - GET /api/orders/ord1 -> returns the order with ID "ord1"
 - POST /api/orders -> creates a new order
 - PUT /api/orders/ord1 -> updates the order with ID "ord1"
 - DELETE /api/orders/ord1 -> deletes the order with ID "ord1"

REST

- The data is typically returned in JSON format
- We can add additional parameters to the data we send with 3 ways:
 - Path parameters: like "ord1", we are telling the api to return the data with this id
 - Query parameters: are added after a "?" at the end of the URL and are "&" separated. For example:
 - GET /api/orders?**sortBy=date&order=asc**
 - This is telling the API to sort the orders by their dates in ascending orders
 - Body: Sending a json body with the request for additional data. For example sending the date of the order in the body of the POST requests

REST

- Okay that was a lot of information...
- For now, know that there are 3 ways to pass arguments to your API call: query parameter, path parameters, and the body
- Use path parameters for IDs to specify the item you are accessing
- Use query parameters for small data like sorting and filtering
- Use body for large data. Typically in JSON
- BTW, GET requests cannot have a body/payload

REST

- Let's try sending some requests to some APIs
- To do that, we will use Postman
- Open it
- Let's play with this [API](#)

Let's design our own API

We want a very simple twitter like API where the user can:

- Upload a post
- Edit a post
- Delete a post
- Read all posts
- Add a comment to a post
- Read post comments

Let's design our own API

- We will design it in using postman mock servers
- Our mock server will return static data
- [Solution](#)

How to integrate frontend to backend

Well, that was fun. Now, how can we make the frontend access our beautiful backend?

- We have a very simple API in javascript called fetch
- We simply give it the http method, the url and other parameters and it calls the API for us
- `fetch('https://.../posts', {method: "POST"})`
- There is one issue though...

How to integrate frontend to backend

- These requests take time!
 - Time for the server to process the request
 - Time for the request and response travelling in the network
- So, we cannot simply block everything waiting for the request to return
- What we want is for the browser to start the request, go on with its business, and when the response arrives it should continue the function
- To do that we use Async/Await

Async/Await

- **async**: A keyword you add before a function declaration. It tells JavaScript, "This function will contain asynchronous operations."
- **await**: A keyword you use inside an async function. It "pauses" the function execution until a Promise resolves, then gives you the result.

Async/Await

- Let's see this in action
- Fork this repo: <https://github.com/mina58/lab-8-web>
- Clone it
- Change the URL in api.js to your mock api
- Implement the functions using await fetch and the API endpoints you created